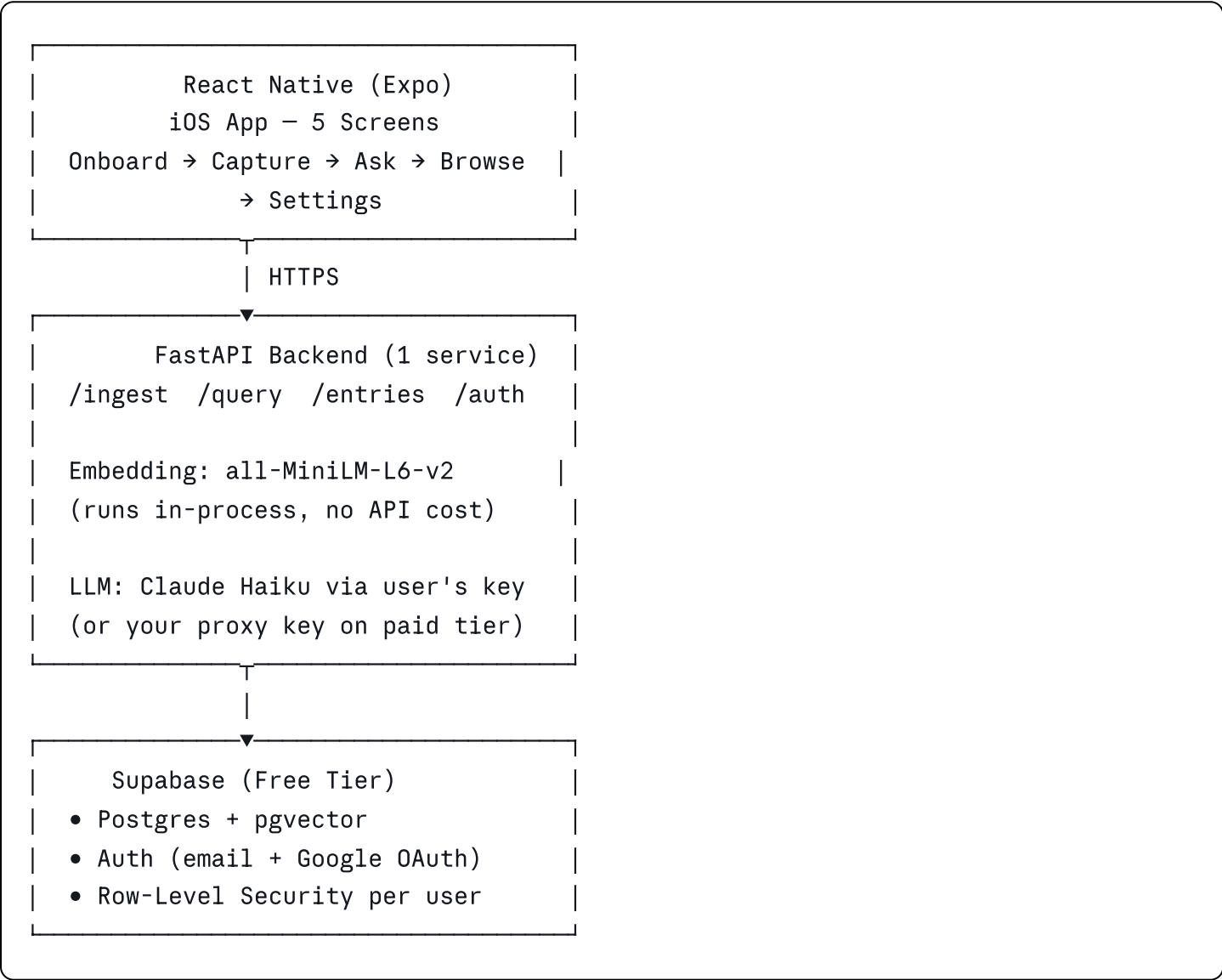


# WisdomBase MVP — 24-Hour Build Plan

## What We're Building

A mobile app where users capture thoughts, ideas, and knowledge, then retrieve them through natural-language conversation with their own past thinking. The core loop: **put knowledge in → ask questions → get your own wisdom back.**

## Architecture (Kept Minimal)



## Tech Stack (Final)

| Layer      | Technology          | Why                                       |
|------------|---------------------|-------------------------------------------|
| Mobile App | React Native + Expo | One codebase, fast iteration, OTA updates |

| Layer        | Technology             | Why                                     |
|--------------|------------------------|-----------------------------------------|
| Navigation   | Expo Router            | File-based routing, simple              |
| State        | Zustand                | Minimal boilerplate, works with RN      |
| Secure Store | expo-secure-store      | API keys in iOS Keychain                |
| Backend      | Python + FastAPI       | Single service, LLM/embedding ecosystem |
| Database     | Supabase (Postgres)    | Free tier, auth, pgvector, RLS          |
| Vectors      | pgvector extension     | No separate vector DB service           |
| Embeddings   | sentence-transformers  | Runs locally in backend, zero API cost  |
| LLM          | Anthropic Claude Haiku | Best cost/quality ratio for RAG         |
| Hosting      | Railway.app            | \$5/mo, simple container deploy         |
| Dev Tool     | VS Code + Claude Code  | AI-assisted coding for speed            |

### Pre-Requisites Checklist (Do This First — 30 min)

- ☐ **Node.js 18+** installed (`node --version`)
- ☐ **Python 3.11+** installed (`python3 --version`)
- ☐ **Expo CLI:** `npm install -g expo-cli` and install **Expo Go** app on your iPhone
- ☐ **VS Code** with extensions: ESLint, Prettier, Python, Expo Tools
- ☐ **Supabase account:** <https://supabase.com> — create a free project
  - Note your: `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`
  - Enable pgvector: run `CREATE EXTENSION IF NOT EXISTS vector;` in SQL editor
- ☐ **Anthropic API key** for testing: <https://console.anthropic.com>
- ☐ **Railway account** (for later deployment): <https://railway.app>
- ☐ **Git repo** initialized

### Recommended: Claude Code (Terminal AI Assistant)

Install Claude Code for VS Code — it will dramatically speed up your build:

```
bash

# Install Claude Code CLI
npm install -g @anthropic-ai/claude-code
```

You can ask Claude Code to generate boilerplate, debug errors, and write tests directly in your terminal while you focus on product decisions.

---

**Database Schema (Set Up in Supabase SQL Editor)**

Run this in your Supabase SQL Editor before writing any code:

```
sql
```

```

-- Enable pgvector
CREATE EXTENSION IF NOT EXISTS vector;

-- Users profile (extends Supabase auth.users)
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  display_name TEXT,
  llm_provider TEXT DEFAULT 'anthropic',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Journal entries / knowledge items
CREATE TABLE public.entries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  title TEXT,
  content TEXT NOT NULL,
  source TEXT DEFAULT 'manual', -- 'manual', 'upload', 'notion', etc.
  tags TEXT[] DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Vector chunks (one entry → multiple chunks)
CREATE TABLE public.chunks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  entry_id UUID NOT NULL REFERENCES public.entries(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  embedding vector(384), -- MiniLM-L6-v2 produces 384-dim vectors
  chunk_index INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Vector similarity search index
CREATE INDEX ON public.chunks
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);

-- Index for filtering by user
CREATE INDEX idx_chunks_user_id ON public.chunks(user_id);
CREATE INDEX idx_entries_user_id ON public.entries(user_id);

-- Row Level Security
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.entries ENABLE ROW LEVEL SECURITY;

```

```
ALTER TABLE public.chunks ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users see own profile"
  ON public.profiles FOR ALL USING (auth.uid() = id);

CREATE POLICY "Users see own entries"
  ON public.entries FOR ALL USING (auth.uid() = user_id);

CREATE POLICY "Users see own chunks"
  ON public.chunks FOR ALL USING (auth.uid() = user_id);

-- Function for vector similarity search
CREATE OR REPLACE FUNCTION match_chunks(
  query_embedding vector(384),
  match_user_id UUID,
  match_count INT DEFAULT 10,
  match_threshold FLOAT DEFAULT 0.3
)
RETURNS TABLE (
  id UUID,
  entry_id UUID,
  content TEXT,
  similarity FLOAT
)
LANGUAGE plpgsql
AS $$
BEGIN
  RETURN QUERY
  SELECT
    chunks.id,
    chunks.entry_id,
    chunks.content,
    1 - (chunks.embedding <=> query_embedding) AS similarity
  FROM public.chunks
  WHERE chunks.user_id = match_user_id
    AND 1 - (chunks.embedding <=> query_embedding) > match_threshold
  ORDER BY chunks.embedding <=> query_embedding
  LIMIT match_count;
END;
$$;
```

---

# Hour-by-Hour Build Schedule

## PHASE 1: Foundation (Hours 0-3)

### Hour 0-1: Project Setup

#### Backend:

```
bash

mkdir wisdombase && cd wisdombase
mkdir backend && cd backend
python3 -m venv venv
source venv/bin/activate
pip install fastapi uvicorn supabase sentence-transformers anthropic python-multipart

# Create structure
touch main.py requirements.txt .env
```

#### Frontend:

```
bash

cd .. # back to wisdombase root
npx create-expo-app@latest frontend --template blank-typescript
cd frontend
npx expo install expo-router expo-secure-store expo-font
npx expo install @supabase/supabase-js
npm install zustand react-native-safe-area-context
npm install react-native-screens expo-status-bar
```

**What you should have:** Two folders — `backend/` and `frontend/` — both running without errors. Backend serves "hello world" on localhost:8000. Frontend shows default Expo screen on your phone via Expo Go.

### Hour 1-2: Authentication

- Set up Supabase Auth in the frontend (email/password to start)
- Three screens: **SignUp**, **SignIn**, **ForgotPassword**
- Auth state managed via Zustand store
- On successful auth, navigate to main app
- Backend receives Supabase JWT in Authorization header, validates it

**Keep it ugly.** No styling yet. Just working auth flows.

### Hour 2-3: Backend Core — Ingest Pipeline

Build two FastAPI endpoints:

**POST /ingest** — receives text content, chunks it, embeds it, stores it:

```
Input: { title, content, source }
Steps: 1. Save entry to entries table
        2. Split content into chunks (~300 words each, overlap 50 words)
        3. Embed each chunk using MiniLM (in-process)
        4. Store chunks + embeddings in chunks table
Output: { entry_id, chunk_count }
```

**POST /query** — receives a question, retrieves relevant chunks, calls LLM:

```
Input: { question, api_key (optional) }
Steps: 1. Embed the question using MiniLM
        2. Call match_chunks() to find top 10 similar chunks
        3. Fetch full entry context for those chunks
        4. Construct prompt with retrieved context
        5. Call Claude Haiku with user's API key
        6. Return synthesized answer + source references
Output: { answer, sources: [{ entry_id, title, snippet, date }] }
```

**Chunking strategy (important — get this right):**

- Split on paragraph boundaries first (double newline)
  - If a paragraph exceeds 300 words, split on sentence boundaries
  - Each chunk carries: the text, its position index, and the entry's creation date as metadata
  - Overlap: include the last sentence of the previous chunk as the first sentence of the next chunk (preserves continuity)
- 

## PHASE 2: Core Experience (Hours 3-8)

### Hour 3-4: Capture Screen

The most important screen in the app. Build it with care.

- Large, clean text input area (full screen, minimal chrome)
- Title field (optional)
- "Save" button that calls POST /ingest
- After save, show brief confirmation, clear the input
- This screen should feel like opening a blank page — inviting, not intimidating

**Implementation notes:**

- Use a `TextInput` with `multiline={true}`, large font (18px)
- Auto-focus on screen mount
- Save to Supabase directly via the JS client for the entry, then trigger backend `/ingest` for embedding (async, don't block UI)

## Hour 4-5: File Upload

- Add a button on the Capture screen: "Upload file"
- Support: .txt, .md files (keep it simple for MVP)
- Use `expo-document-picker` to select files
- Read file content, send to `/ingest` endpoint
- Show upload progress

**Skip PDF parsing for MVP.** It's a rabbit hole. Accept plain text and markdown only. Tell users to copy-paste from other formats.

## Hour 5-7: Ask Screen (The Core Magic)

This is where the product lives or dies. Spend the most time here.

- Chat-style interface (messages between user and the app)
- User types a question at the bottom
- App shows a thinking/loading state
- Response appears as a message with:
  - The synthesized answer (from LLM)
  - Expandable "source" cards showing which entries were referenced
  - Each source card shows: entry title, date, relevant snippet
- Conversation continues — user can ask follow-ups
- Conversation history is sent with each request (last 5 turns)

**The prompt template (critical):**



You are the user's personal knowledge assistant. You have access to their past journal entries, ideas, and notes. Your role is to help them retrieve and synthesize their own thinking.

Rules:

- Only reference information from the provided context
- If the context doesn't contain relevant information, say so honestly
- Quote their own words when relevant (they wrote this, remind them)
- Note the dates of relevant entries so they have temporal context
- Be conversational, warm, and concise
- If you see patterns across entries, point them out

Context from the user's knowledge base:

---

{retrieved\_chunks\_with\_dates\_and\_titles}

---

Conversation so far:

{conversation\_history}

User's question: {question}

## Hour 7-8: Browse Screen

- Simple chronological list of all entries
- Each item shows: title (or first line), date, preview snippet
- Tap to view full entry
- Pull-to-refresh
- Search bar at top (basic text search via Supabase full-text, not vector)
- Swipe to delete

---

## PHASE 3: Polish & Completion (Hours 8-14)

### Hour 8-9: Settings Screen

- Display user email
- API Key input field (stored in expo-secure-store)
  - With clear explanation: "Your API key is stored securely on your device and is never saved on our servers"
- "Export my data" button (downloads all entries as JSON)
- "Delete my account" button
- App version number

## Hour 9-10: Onboarding Flow

Build the "brain dump" onboarding that solves the cold-start problem:

- Welcome screen explaining the concept (2-3 sentences max)
- Screen 1: "What's the biggest decision you're facing right now?"
- Screen 2: "What's a lesson you learned the hard way?"
- Screen 3: "What's an idea you keep coming back to?"
- Screen 4: "What do you know about your work/field that most don't?"
- Screen 5: "What advice would you give your younger self?"
- Each screen has a text input, "Skip" and "Next" buttons
- All non-empty responses get ingested as entries
- After completion: "You've planted 5 seeds of wisdom. Start asking questions or keep adding."

## Hour 10-12: Styling & UX Polish

Now make it beautiful. Not before.

- Color palette: warm, muted tones (not tech-blue, not clinical)
- Typography: clean serif for entry content, sans-serif for UI
- Animations: subtle fade-ins for retrieved results
- Loading states: meaningful (not just spinners — "Searching your memories..." "Connecting the dots...")
- Empty states: encouraging (not "no entries found" but "Your knowledge base is growing. Keep adding thoughts.")
- Haptic feedback on save (expo-haptics)

## Hour 12-13: Error Handling & Edge Cases

- No internet connection → show cached entries, disable Ask
- Invalid/expired API key → clear error message with link to get one
- Empty query → gentle prompt to ask something specific
- Very long entries → chunk correctly without breaking mid-word
- LLM rate limit hit → queue and retry with backoff

## Hour 13-14: Backend Deployment

- Deploy FastAPI to Railway:

```
bash
```

```
# In backend/  
# Create Dockerfile  
# Create railway.toml  
# railway login && railway up
```

- Set environment variables in Railway dashboard
  - Test all endpoints against production
  - Update frontend to point to production backend URL
- 

## PHASE 4: Ship to Phone (Hours 14-16)

### Hour 14-15: Expo Development Build

To test on your actual iPhone (beyond Expo Go):

```
bash  
  
# Install EAS CLI  
npm install -g eas-cli  
  
# Login to Expo account (create one if needed)  
eas login  
  
# Configure the build  
eas build:configure  
  
# Create a development build for iOS  
eas build --platform ios --profile development  
  
# OR for faster iteration, use Expo Go:  
# Just run: npx expo start --tunnel  
# and scan the QR code with your iPhone camera
```

**For the first 24 hours, Expo Go is perfectly fine.** You don't need a full native build yet. Expo Go lets you run the app on your phone immediately by scanning a QR code. The only limitation is you can't use certain native modules, but nothing in our MVP requires them.

### Hour 15-16: Testing on Device

- Run through complete flow on your actual phone
- Sign up → onboarding brain dump → capture an entry → ask a question
- Test with real content (your actual ideas and thoughts)
- Fix anything that feels wrong on a real device (keyboard handling, scroll behavior, text sizes)

---

## File Structure

```
wisdombase/
├── backend/
│   ├── main.py           # FastAPI app, all routes
│   ├── chunker.py        # Text splitting logic
│   ├── embedder.py       # Sentence-transformers wrapper
│   ├── rag.py            # Retrieval + prompt construction
│   ├── llm.py            # Anthropic API calls
│   ├── auth.py           # JWT validation middleware
│   ├── requirements.txt
│   ├── Dockerfile
│   └── .env
├── frontend/
│   ├── app/
│   │   ├── _layout.tsx   # Root layout + auth guard
│   │   ├── index.tsx     # Entry point → redirect
│   │   └── (auth)/
│   │       ├── sign-in.tsx
│   │       ├── sign-up.tsx
│   │       └── _layout.tsx
│   │   ├── (app)/
│   │       ├── _layout.tsx # Tab navigation
│   │       ├── capture.tsx # Add entries
│   │       ├── ask.tsx     # Query your knowledge
│   │       ├── browse.tsx  # View all entries
│   │       └── settings.tsx
│   │   └── onboarding/
│   │       └── index.tsx   # Brain dump flow
│   ├── components/
│   │   ├── EntryCard.tsx
│   │   ├── ChatMessage.tsx
│   │   ├── SourceCard.tsx
│   │   └── LoadingDots.tsx
│   ├── lib/
│   │   ├── supabase.ts   # Supabase client init
│   │   ├── api.ts        # Backend API calls
│   │   └── secureStore.ts # API key management
│   ├── stores/
│   │   ├── authStore.ts  # Zustand auth state
│   │   └── entryStore.ts # Zustand entries state
│   └── app.json
```

package.json

README.md

## Cost Breakdown (Monthly)

| Item               | Cost           | Notes                                |
|--------------------|----------------|--------------------------------------|
| Supabase Free Tier | \$0            | 500MB DB, 50K auth users             |
| Railway (backend)  | ~\$5           | Small container                      |
| Embedding model    | \$0            | Runs in backend process              |
| Claude Haiku API   | ~\$0.001/query | User pays via own key, or you absorb |
| Expo / EAS         | \$0            | Free tier for dev builds             |
| Total (your cost)  | ~\$5/month     | Until significant scale              |

## Success Criteria for the 24-Hour Sprint

By hour 16, you should be able to:

1. Open the app on your iPhone
2. Sign up with email
3. Go through the brain dump onboarding (5 prompts)
4. Add a new thought/idea manually
5. Upload a .txt file with past notes
6. Ask "What are the themes in my thinking?" and get a meaningful answer
7. See your entries listed chronologically
8. Input your Anthropic API key in settings

If all 8 work, you have an MVP. Everything else is iteration.

## What to Build NEXT (After the 24 Hours)

In priority order:

1. **Proxy API key model** — add your own Anthropic key on backend, offer \$3-5/month subscription so non-technical users skip BYOK
  2. **Notion integration** — OAuth flow + Notion API to import pages
  3. **Google Drive integration** — import Google Docs
  4. **Proactive insights** — weekly digest email: "Here's what patterns I see in your recent thinking"
  5. **Tags & collections** — let users organize entries
  6. **Voice capture** — record a thought, transcribe via Whisper, ingest as entry (very natural on mobile)
  7. **Share a wisdom card** — export a single insight as a beautiful image for sharing (growth loop)
  8. **Web app** — Next.js, sharing auth and backend with mobile
- 

## Development Tips for Speed

**Use Claude Code aggressively.** For each component or endpoint:

1. Describe what you want in plain English
2. Let Claude Code generate the first draft
3. Review, adjust, move on
4. Don't perfect anything — get it working, then iterate

**Test the Ask flow early.** Even before the UI is pretty, make sure the backend /query endpoint returns good answers. This is the product. If retrieval quality is poor, nothing else matters.

**Don't style until hour 10.** Ugly-but-working beats pretty-but-broken. The temptation to make things look good early is the #1 time killer.

**Keep a bug list, don't fix immediately.** If something non-critical breaks, write it down and keep building forward. Fix in Phase 3.

**Commit after each phase.** You want rollback points.